Algorithm Patterns – All 18 patterns. Print or Save as PDF from the browser (Ctrl+P / Cmd+P).

Sliding Window Pattern

Sliding Window

📖 Concept

The Sliding Window pattern is used to solve problems involving subarrays or substrings of a fixed or variable size. Instead of recalculating everything for each window, we efficiently slide the window by adding one element and removing another.

Key Idea: Maintain a window that slides through the array/string, updating the window state incrementally rather than recalculating from scratch.

🎯 Techniques to Master

- **Fixed Size Window:** Window size is constant (e.g., find max sum of k consecutive elements)
- **Variable Size Window:** Window size changes based on conditions (e.g., smallest subarray with sum ≥ target)
- **Two Pointers:** Use left and right pointers to expand/contract the window
- **Hash Map Tracking:** Use hash maps to track character/element frequencies in the window
- **Window State:** Maintain running sum, count, or other metrics as the window moves
- **Expansion Strategy:** Expand right pointer until condition met, then contract left pointer

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(n)	Each element is visited at most twice (once by left pointer, once by right pointer)
Space Complexity	O(k) or O(1)	O(k) if using hash map for window tracking, O(1) for simple sum/count tracking

📚 Boilerplate Code

Fixed Size Window:

```
def fixed_window(arr, k):
    """
    Find maximum sum of subarray of size k
    """
    if len(arr) < k:
        return 0

    # Calculate sum of first window
    window_sum = sum(arr[:k])
    max_sum = window_sum

    # Slide the window
    for i in range(k, len(arr)):
        # Remove leftmost element, add rightmost element
        window_sum = window_sum - arr[i - k] + arr[i]
        max_sum = max(max_sum, window_sum)

    return max_sum

# TC: O(n)
# SC: O(1)
```

Variable Size Window:

```
def variable_window(arr, target):
    """
    Find minimum length subarray with sum ≥ target
    """
    left = 0
    window_sum = 0
    min_len = float('inf')

    for right in range(len(arr)):
        # Expand window
        window_sum += arr[right]

        # Contract window while condition is met
        while window_sum ≥ target:
            min_len = min(min_len, right - left + 1)
            window_sum -= arr[left]
            left += 1

    return min_len if min_len ≠ float('inf') else 0

# TC: O(n)
# SC: O(1)
```

Sliding Window with Hash Map:

```
def longest_substring_k_distinct(s, k):
    """
    Find longest substring with at most k distinct characters
    """
    char_count = {}
    left = 0
    max_len = 0

    for right in range(len(s)):
        # Add character to window
        char_count[s[right]] = char_count.get(s[right], 0) + 1

        # Shrink window if more than k distinct chars
        while len(char_count) > k:
            char_count[s[left]] -= 1
            if char_count[s[left]] == 0:
                del char_count[s[left]]
            left += 1

        max_len = max(max_len, right - left + 1)

    return max_len

# TC: O(n)
# SC: O(k)
```

📝 Demonstrative Problems

Problem 1: Maximum Sum Subarray of Size K

Description: Given an array of integers and a number k, find the maximum sum of any contiguous subarray of size k.

Example: arr = [2, 1, 5, 1, 3, 2], k = 3 → Output: 9 (subarray [5, 1, 3])

```
def max_sum_subarray(arr, k):
    if len(arr) < k:
        return 0

    window_sum = sum(arr[:k])
    max_sum = window_sum

    for i in range(k, len(arr)):
        window_sum = window_sum - arr[i - k] + arr[i]
        max_sum = max(max_sum, window_sum)

    return max_sum
```

Problem 2: Longest Substring Without Repeating Characters

Description: Find the length of the longest substring without repeating characters.

Example: "abcabcbb" → Output: 3 ("abc")

```
def length_of_longest_substring(s):
    char_index = {}
    left = 0
    max_len = 0

    for right in range(len(s)):
        # If character seen and within current window
        if s[right] in char_index and char_index[s[right]] ≥ left:
            left = char_index[s[right]] + 1

        char_index[s[right]] = right
        max_len = max(max_len, right - left + 1)

    return max_len
```

Problem 3: Minimum Window Substring

Description: Find the minimum window in string s that contains all characters of string t.

Example: s = "ADOBECODEBANC", t = "ABC" → Output: "BANC"

```
def min_window(s, t):
    if not s or not t:
        return ""

    # Count characters in t
    need = {}
    for char in t:
        need[char] = need.get(char, 0) + 1

    left = 0
    valid = 0  # Number of unique characters in current window matching t
    window = {}
    start = 0
    min_len = float('inf')

    for right in range(len(s)):
        c = s[right]
        if c in need:
            window[c] = window.get(c, 0) + 1
            if window[c] == need[c]:
                valid += 1

        # Try to shrink window
        while valid == len(need):
            if right - left + 1 < min_len:
                start = left
                min_len = right - left + 1

            d = s[left]
            if d in need:
                if window[d] == need[d]:
                    valid -= 1
                window[d] -= 1
            left += 1

    return "" if min_len == float('inf') else s[start:start + min_len]
```

Two Pointers Pattern

Two Pointers

📖 Concept

The Two Pointers pattern uses two pointers that traverse the data structure in a coordinated way. This pattern is highly efficient for solving problems involving arrays, strings, or linked lists.

Key Idea: Use two pointers moving at different speeds or from different directions to efficiently solve problems without nested loops.

🎯 Techniques to Master

- **Opposite Ends:** Start pointers at opposite ends and move them toward each other
- **Fast & Slow:** Use two pointers moving at different speeds (Floyd's cycle detection)
- **Same Direction:** Both pointers move in the same direction at different speeds
- **Sliding Window:** Two pointers define a window that slides through the array
- **Partitioning:** Use pointers to partition array into sections
- **Cycle Detection:** Fast pointer catches slow pointer if cycle exists

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(n)	Each element is visited at most once by each pointer
Space Complexity	O(1)	Only using a constant amount of extra space for pointers

💻 Boilerplate Code

Opposite Ends (Two Sum in Sorted Array):

```
def two_sum_sorted(arr, target):
    """
    Find two numbers that add up to target in sorted array
    """
    left, right = 0, len(arr) - 1

    while left < right:
        current_sum = arr[left] + arr[right]
        if current_sum == target:
            return [left, right]
        elif current_sum < target:
            left += 1
        else:
            right -= 1

    return []

# TC: O(n)
# SC: O(1)
```

Fast & Slow (Cycle Detection):

```
def has_cycle(head):
    """
    Detect cycle in linked list using Floyd's algorithm
    """
    if not head or not head.next:
        return False

    slow = fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True

    return False

# TC: O(n)
# SC: O(1)
```

Same Direction (Remove Duplicates):

```
def remove_duplicates(arr):
    """
    Remove duplicates in-place from sorted array
    """
    if not arr:
        return 0

    write_ptr = 1

    for read_ptr in range(1, len(arr)):
        if arr[read_ptr] ≠ arr[read_ptr - 1]:
            arr[write_ptr] = arr[read_ptr]
            write_ptr += 1

    return write_ptr

# TC: O(n)
# SC: O(1)
```

📝 Demonstrative Problems

Problem 1: Two Sum II - Input Array Is Sorted

Description: Given a sorted array, find two numbers that add up to a specific target.

Example: numbers = [2, 7, 11, 15], target = 9 → Output: [1, 2]

```
def two_sum(numbers, target):
    left, right = 0, len(numbers) - 1

    while left < right:
        current_sum = numbers[left] + numbers[right]
        if current_sum == target:
            return [left + 1, right + 1]  # 1-indexed
        elif current_sum < target:
            left += 1
        else:
            right -= 1

    return []
```

Problem 2: Linked List Cycle

Description: Determine if a linked list has a cycle using O(1) space.

Example: head = [3,2,0,-4], pos = 1 → Output: True

```
def has_cycle(head):
    if not head or not head.next:
        return False

    slow = fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True

    return False
```

Problem 3: Container With Most Water

Description: Find two lines that together with the x-axis forms a container that holds the most water.

Example: height = [1,8,6,2,5,4,8,3,7] → Output: 49

```
def max_area(height):
    left, right = 0, len(height) - 1
    max_water = 0

    while left < right:
        width = right - left
        current_area = min(height[left], height[right]) * width
        max_water = max(max_water, current_area)

        # Move pointer with smaller height
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1

    return max_water
```

Merge Intervals Pattern

Merge Intervals

📖 Concept

The Merge Intervals pattern is used to solve problems involving overlapping intervals. The key is to sort intervals by their start time and then merge or find relationships between them.

Key Idea: Sort intervals by start time, then iterate through them, merging overlapping intervals or finding gaps/conflicts.

🎯 Techniques to Master

- **Sort First:** Always sort intervals by start time (or end time for specific problems)
- **Overlap Detection:** Two intervals overlap if start[i] <= end[j] and start[j] <= end[i]
- **Merge Strategy:** Compare current interval with last merged interval
- **Gap Finding:** Identify gaps between non-overlapping intervals
- **Conflict Detection:** Find intervals that conflict with each other
- **Insert Intervals:** Insert new interval and merge if necessary

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(n log n)	Sorting takes O(n log n), merging takes O(n)
Space Complexity	O(n)	Space needed for storing merged intervals

💻 Boilerplate Code

Basic Merge Intervals:

```
def merge_intervals(intervals):
    """
    Merge overlapping intervals
    """
    if not intervals:
        return []

    # Sort by start time
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]

    for current in intervals[1:]:
        last = merged[-1]
        # If current overlaps with last, merge them
        if current[0] ≤ last[1]:
            merged[-1] = [last[0], max(last[1], current[1])]
        else:
            merged.append(current)

    return merged

# TC: O(n log n)
# SC: O(n)
```

Insert Interval:

```
def insert_interval(intervals, new_interval):
    """
    Insert new interval and merge if necessary
    """
    result = []
    i = 0

    # Add all intervals before new_interval
    while i < len(intervals) and intervals[i][1] < new_interval[0]:
        result.append(intervals[i])
        i += 1

    # Merge overlapping intervals
    while i < len(intervals) and intervals[i][0] ≤ new_interval[1]:
        new_interval[0] = min(new_interval[0], intervals[i][0])
        new_interval[1] = max(new_interval[1], intervals[i][1])
        i += 1

    result.append(new_interval)

    # Add remaining intervals
    result.extend(intervals[i:])
    return result

# TC: O(n)
# SC: O(n)
```

📝 Demonstrative Problems

Problem 1: Merge Intervals

Description: Given an array of intervals, merge all overlapping intervals.

Example: intervals = [[1,3],[2,6],[8,10],[15,18]] → Output: [[1,6],[8,10],[15,18]]

```
def merge(intervals):
    if not intervals:
        return []

    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]

    for current in intervals[1:]:
        last = merged[-1]
        if current[0] ≤ last[1]:
            merged[-1] = [last[0], max(last[1], current[1])]
        else:
            merged.append(current)

    return merged
```

Problem 2: Insert Interval

Description: Insert a new interval into a sorted list of non-overlapping intervals and merge if necessary.

Example: intervals = [[1,3],[6,9]], newInterval = [2,5] → Output: [[1,5],[6,9]]

```
def insert(intervals, new_interval):
    result = []
    i = 0

    while i < len(intervals) and intervals[i][1] < new_interval[0]:
        result.append(intervals[i])
        i += 1

    while i < len(intervals) and intervals[i][0] ≤ new_interval[1]:
        new_interval[0] = min(new_interval[0], intervals[i][0])
        new_interval[1] = max(new_interval[1], intervals[i][1])
        i += 1

    result.append(new_interval)
    result.extend(intervals[i:])
    return result
```

Problem 3: Meeting Rooms II

Description: Find the minimum number of conference rooms required.

Example: intervals = [[0,30],[5,10],[15,20]] → Output: 2

```
def min_meeting_rooms(intervals):
    if not intervals:
        return 0

    start_times = sorted([i[0] for i in intervals])
    end_times = sorted([i[1] for i in intervals])

    start_ptr = end_ptr = 0
    rooms = 0
    max_rooms = 0

    while start_ptr < len(intervals):
        if start_times[start_ptr] < end_times[end_ptr]:
            rooms += 1
            start_ptr += 1
        else:
            rooms -= 1
            end_ptr += 1
        max_rooms = max(max_rooms, rooms)

    return max_rooms
```

Cyclic Sort Pattern

Cyclic Sort

📖 Concept

Cyclic Sort is an in-place sorting algorithm that works well when dealing with arrays containing numbers in a given range. It places each number at its correct index.

Key Idea: For an array of n elements with values in range [0, n-1] or [1, n], each value should be at index equal to its value. We swap elements to their correct positions.

🎯 Techniques to Master

- **Range-Based Sorting:** Works best when numbers are in a known range
- **Index Mapping:** Value at index i should be i (or i+1 for 1-based)
- **Swap Strategy:** Swap current element to its correct position
- **Missing Number Detection:** After sorting, find index where value != index
- **Duplicate Detection:** Identify duplicates during sorting process
- **Boundary Checks:** Ensure indices are within valid range before swapping

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(n)	Each element is swapped at most once to its correct position
Space Complexity	O(1)	Sorting is done in-place, only using constant extra space

📚 Boilerplate Code

Basic Cyclic Sort (0 to n-1):

```
def cyclic_sort(arr):
    """
    Sort array where numbers are in range [0, n-1]
    """
    i = 0
    while i < len(arr):
        # If number is at correct position, move to next
        if arr[i] == i:
            i += 1
        else:
            # Swap to correct position
            arr[i], arr[arr[i]] = arr[arr[i]], arr[i]

    return arr

# TC: O(n)
# SC: O(1)
```

Cyclic Sort (1 to n):

```
def cyclic_sort_1_to_n(arr):
    """
    Sort array where numbers are in range [1, n]
    """
    i = 0
    while i < len(arr):
        correct_pos = arr[i] - 1
        if arr[i] != arr[correct_pos]:
            arr[i], arr[correct_pos] = arr[correct_pos], arr[i]
        else:
            i += 1

    return arr

# TC: O(n)
# SC: O(1)
```

Find Missing Number:

```
def find_missing_number(arr):
    """
    Find missing number in array containing n distinct numbers from range [0, n]
    """
    i = 0
    while i < len(arr):
        if arr[i] < len(arr) and arr[i] ≠ i:
            arr[i], arr[arr[i]] = arr[arr[i]], arr[i]
        else:
            i += 1

    # Find first index where value ≠ index
    for i in range(len(arr)):
        if arr[i] ≠ i:
            return i

    return len(arr)  # n is missing

# TC: O(n)
# SC: O(1)
```

📝 Demonstrative Problems

Problem 1: Find Missing Number

Description: Given an array containing n distinct numbers from range [0, n], find the missing number.

Example: nums = [3, 0, 1] → Output: 2

```
def missing_number(nums):
    i = 0
    while i < len(nums):
        if nums[i] < len(nums) and nums[i] ≠ i:
            nums[i], nums[nums[i]] = nums[nums[i]], nums[i]
        else:
            i += 1

    for i in range(len(nums)):
        if nums[i] ≠ i:
            return i

    return len(nums)
```

Problem 2: Find All Duplicates

Description: Find all duplicates in an array where numbers are in range [1, n].

Example: nums = [4,3,2,7,8,2,3,1] → Output: [2,3]

```
def find_duplicates(nums):
    i = 0
    while i < len(nums):
        correct_pos = nums[i] - 1
        if nums[i] ≠ nums[correct_pos]:
            nums[i], nums[correct_pos] = nums[correct_pos], nums[i]
        else:
            i += 1

    duplicates = []
    for i in range(len(nums)):
        if nums[i] ≠ i + 1:
            duplicates.append(nums[i])

    return duplicates
```

Problem 3: Find All Missing Numbers

Description: Find all numbers missing from an array containing n numbers from range [1, n].

Example: nums = [4,3,2,7,8,2,3,1] → Output: [5,6]

```
def find_missing_numbers(nums):
    i = 0
    while i < len(nums):
        correct_pos = nums[i] - 1
        if nums[i] ≠ nums[correct_pos]:
            nums[i], nums[correct_pos] = nums[correct_pos], nums[i]
        else:
            i += 1

    missing = []
    for i in range(len(nums)):
        if nums[i] ≠ i + 1:
            missing.append(i + 1)

    return missing
```

Linked List Pattern

Linked List

📖 Concept

Linked List problems involve manipulating nodes connected via pointers. Common operations include reversal, cycle detection, finding middle node, and merging lists.

Key Idea: Use pointer manipulation to traverse, modify, or restructure linked lists without using extra space for arrays.

🎯 Techniques to Master

- **Two Pointers:** Fast & slow pointers for cycle detection, finding middle
- **In-place Reversal:** Reverse linked list by changing pointers
- **Dummy Head:** Use dummy node to simplify edge cases
- **Pointer Manipulation:** Carefully update next pointers to avoid losing references
- **Cycle Detection:** Floyd's algorithm (tortoise and hare)
- **Merge Lists:** Combine two sorted lists efficiently

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(n)	Most operations require single pass through the list
Space Complexity	O(1)	In-place operations use only constant extra space

💻 Boilerplate Code

Reverse Linked List:

```
def reverse_list(head):
    """
    Reverse a linked list in-place
    """
    prev = None
    current = head

    while current:
        next_node = current.next
        current.next = prev
        prev = current
        current = next_node

    return prev

# TC: O(n)
# SC: O(1)
```

Find Middle Node:

```
def find_middle(head):
    """
    Find middle node using fast and slow pointers
    """
    slow = fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    return slow

# TC: O(n)
# SC: O(1)
```

Cycle Detection:

```
def has_cycle(head):
    """
    Detect cycle using Floyd's algorithm
    """
    if not head or not head.next:
        return False

    slow = fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True

    return False

# TC: O(n)
# SC: O(1)
```

📝 Demonstrative Problems

Problem 1: Reverse Linked List

Description: Reverse a singly linked list.

Example: 1->2->3->4->5 → 5->4->3->2->1

```
def reverse_list(head):
    prev = None
    current = head

    while current:
        next_node = current.next
        current.next = prev
        prev = current
        current = next_node

    return prev
```

Problem 2: Remove Nth Node From End

Description: Remove the nth node from the end of the list.

Example: head = [1,2,3,4,5], n = 2 → [1,2,3,5]

```
def remove_nth_from_end(head, n):
    dummy = ListNode(0)
    dummy.next = head

    first = second = dummy

    # Move first pointer n+1 steps ahead
    for _ in range(n + 1):
        first = first.next

    # Move both pointers until first reaches end
    while first:
        first = first.next
        second = second.next

    # Remove the node
    second.next = second.next.next

    return dummy.next
```

Problem 3: Merge Two Sorted Lists

Description: Merge two sorted linked lists into one sorted list.

Example: list1 = [1,2,4], list2 = [1,3,4] → [1,1,2,3,4,4]

```
def merge_two_lists(list1, list2):
    dummy = ListNode(0)
    current = dummy

    while list1 and list2:
        if list1.val ≤ list2.val:
            current.next = list1
            list1 = list1.next
        else:
            current.next = list2
            list2 = list2.next
        current = current.next

    # Attach remaining nodes
    current.next = list1 if list1 else list2

    return dummy.next
```

Tree BFS Pattern

Tree BFS (Breadth-First Search)

📖 Concept

BFS traverses a tree level by level, visiting all nodes at the current depth before moving to the next level. It uses a queue to process nodes.

Key Idea: Process nodes level by level using a queue. Start with root, add children to queue, process each level completely before moving to next.

🎯 Techniques to Master

- **Queue-Based:** Use queue (deque) to maintain nodes at current level
- **Level Tracking:** Process all nodes at current level before moving to next
- **Level Order Traversal:** Return nodes grouped by level
- **Zigzag Traversal:** Alternate direction at each level
- **Right Side View:** Track last node at each level
- **Level Size:** Track number of nodes at each level for processing

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(n)	Visit each node exactly once
Space Complexity	O(w)	w is maximum width of tree (queue size at widest level)

💻 Boilerplate Code

Basic BFS:

```
from collections import deque

def bfs(root):
    """
    Basic BFS traversal
    """
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        node = queue.popleft()
        result.append(node.val)

        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)

    return result

# TC: O(n)
# SC: O(w)
```

Level Order Traversal:

```
def level_order(root):
    """
    Return nodes grouped by level
    """
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)
        level = []

        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)

            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        result.append(level)

    return result

# TC: O(n)
# SC: O(w)
```

Zigzag Level Order:

```
def zigzag_level_order(root):
    """
    Traverse tree in zigzag pattern
    """
    if not root:
        return []

    result = []
    queue = deque([root])
    left_to_right = True

    while queue:
        level_size = len(queue)
        level = deque()

        for _ in range(level_size):
            node = queue.popleft()

            if left_to_right:
                level.append(node.val)
            else:
                level.appendleft(node.val)

            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        result.append(list(level))
        left_to_right = not left_to_right

    return result

# TC: O(n)
# SC: O(w)
```

📝 Demonstrative Problems

Problem 1: Binary Tree Level Order Traversal

Description: Return the level order traversal of a binary tree.

Example: [3,9,20,null,null,15,7] → [[3],[9,20],[15,7]]

```
def level_order(root):
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)
        level = []

        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)

            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        result.append(level)

    return result
```

Problem 2: Binary Tree Right Side View

Description: Return the values of nodes visible from the right side.

Example: [1,2,3,null,5,null,4] → [1,3,4]

```
def right_side_view(root):
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)

        for i in range(level_size):
            node = queue.popleft()

            # Last node at this level
            if i == level_size - 1:
                result.append(node.val)

            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

    return result
```

Problem 3: Average of Levels in Binary Tree

Description: Return the average value of nodes at each level.

Example: [3,9,20,null,null,15,7] → [3.0, 14.5, 11.0]

```
def average_of_levels(root):
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        level_size = len(queue)
        level_sum = 0

        for _ in range(level_size):
            node = queue.popleft()
            level_sum += node.val

            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        result.append(level_sum / level_size)

    return result
```

Tree DFS Pattern

Tree DFS (Depth-First Search)

📖 Concept

DFS traverses a tree by going as deep as possible before backtracking. It uses recursion or a stack to explore paths.

Key Idea: Explore one path completely before backtracking. Three main orders: Pre-order (root-left-right), In-order (left-root-right), Post-order (left-right-root).

🎯 Techniques to Master

- **Recursive DFS:** Use function calls to naturally traverse depth-first
- **Iterative DFS:** Use stack to simulate recursion
- **Pre-order:** Process root before children (root-left-right)
- **In-order:** Process left, then root, then right (for BST gives sorted order)
- **Post-order:** Process children before root (left-right-root)
- **Path Tracking:** Maintain current path while traversing

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(n)	Visit each node exactly once
Space Complexity	O(h)	h is height of tree (recursion stack or explicit stack)

💻 Boilerplate Code

Pre-order Traversal (Recursive):

```
def preorder_traversal(root):
    """
    Pre-order: root → left → right
    """
    result = []

    def dfs(node):
        if not node:
            return
        result.append(node.val)
        dfs(node.left)
        dfs(node.right)

    dfs(root)
    return result

# TC: O(n)
# SC: O(h)
```

In-order Traversal (Recursive):

```
def inorder_traversal(root):
    """
    In-order: left → root → right
    """
    result = []

    def dfs(node):
        if not node:
            return
        dfs(node.left)
        result.append(node.val)
        dfs(node.right)

    dfs(root)
    return result

# TC: O(n)
# SC: O(h)
```

Post-order Traversal (Recursive):

```
def postorder_traversal(root):
    """
    Post-order: left → right → root
    """
    result = []

    def dfs(node):
        if not node:
            return
        dfs(node.left)
        dfs(node.right)
        result.append(node.val)

    dfs(root)
    return result

# TC: O(n)
# SC: O(h)
```

Iterative DFS (Pre-order):

```
def preorder_iterative(root):
    """
    Pre-order using stack
    """
    if not root:
        return []

    result = []
    stack = [root]

    while stack:
        node = stack.pop()
        result.append(node.val)

        # Push right first, then left (so left is processed first)
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)

    return result

# TC: O(n)
# SC: O(h)
```

📝 Demonstrative Problems

Problem 1: Maximum Depth of Binary Tree

Description: Find the maximum depth of a binary tree.

Example: [3,9,20,null,null,15,7] → 3

```
def max_depth(root):
    if not root:
        return 0

    left_depth = max_depth(root.left)
    right_depth = max_depth(root.right)

    return 1 + max(left_depth, right_depth)
```

Problem 2: Path Sum

Description: Check if there exists a root-to-leaf path with given sum.

Example: root = [5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum = 22 → True

```
def has_path_sum(root, target_sum):
    if not root:
        return False

    if not root.left and not root.right:
        return root.val == target_sum

    remaining = target_sum - root.val
    return (has_path_sum(root.left, remaining) or
            has_path_sum(root.right, remaining))
```

Problem 3: Binary Tree Maximum Path Sum

Description: Find the maximum path sum (path can start and end at any node).

Example: [-10,9,20,null,null,15,7] → 42

```
def max_path_sum(root):
    max_sum = float('-inf')

    def dfs(node):
        nonlocal max_sum
        if not node:
            return 0

        left_sum = max(0, dfs(node.left))
        right_sum = max(0, dfs(node.right))

        # Path through current node
        current_path = node.val + left_sum + right_sum
        max_sum = max(max_sum, current_path)

        # Return max path from this node (can only use one branch)
        return node.val + max(left_sum, right_sum)

    dfs(root)
    return max_sum
```

Topological Sort Pattern

Topological Sort

📖 Concept

Topological Sort orders nodes in a directed acyclic graph (DAG) such that for every edge (u, v), u appears before v in the ordering.

Key Idea: Process nodes with no incoming edges first, then remove their outgoing edges and repeat. Used for dependency resolution, course scheduling, build systems.

🎯 Techniques to Master

- **Kahn's Algorithm:** Use in-degree counting and queue
- **DFS-based:** Use DFS with post-order processing
- **In-degree Tracking:** Count incoming edges for each node
- **Cycle Detection:** Detect if graph has cycles (no valid topological order)
- **Dependency Resolution:** Process dependencies before dependents
- **Queue Management:** Process nodes with zero in-degree

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(V + E)	V = vertices, E = edges. Visit each node and edge once
Space Complexity	O(V)	Space for in-degree array, queue, and result

📚 Boilerplate Code

Kahn's Algorithm (BFS-based):

```
from collections import deque

def topological_sort(num_courses, prerequisites):
    """
    Kahn's algorithm using in-degree counting
    """
    # Build graph and in-degree count
    graph = [[] for _ in range(num_courses)]
    in_degree = [0] * num_courses

    for course, prereq in prerequisites:
        graph[prereq].append(course)
        in_degree[course] += 1

    # Start with nodes having no incoming edges
    queue = deque([i for i in range(num_courses) if in_degree[i] == 0])
    result = []

    while queue:
        node = queue.popleft()
        result.append(node)

        # Remove edges and update in-degrees
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    # Check if all courses were processed (no cycle)
    return result if len(result) == num_courses else []

# TC: O(V + E)
# SC: O(V)
```

DFS-based Topological Sort:

```
def topological_sort_dfs(num_courses, prerequisites):
    """
    DFS-based topological sort
    """
    graph = [[] for _ in range(num_courses)]
    for course, prereq in prerequisites:
        graph[prereq].append(course)

    WHITE, GRAY, BLACK = 0, 1, 2
    color = [WHITE] * num_courses
    result = []

    def dfs(node):
        if color[node] == GRAY:  # Cycle detected
            return False
        if color[node] == BLACK:  # Already processed
            return True

        color[node] = GRAY
        for neighbor in graph[node]:
            if not dfs(neighbor):
                return False

        color[node] = BLACK
        result.append(node)
        return True

    for i in range(num_courses):
        if color[i] == WHITE:
            if not dfs(i):
                return []  # Cycle found

    return result[::-1]  # Reverse for correct order

# TC: O(V + E)
# SC: O(V)
```

📝 Demonstrative Problems

Problem 1: Course Schedule

Description: Determine if you can finish all courses given prerequisites.

Example: numCourses = 2, prerequisites = [[1,0]] → True

```
def can_finish(num_courses, prerequisites):
    graph = [[] for _ in range(num_courses)]
    in_degree = [0] * num_courses

    for course, prereq in prerequisites:
        graph[prereq].append(course)
        in_degree[course] += 1

    queue = deque([i for i in range(num_courses) if in_degree[i] == 0])
    count = 0

    while queue:
        node = queue.popleft()
        count += 1
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    return count == num_courses
```

Problem 2: Course Schedule II

Description: Find the order of courses to finish all courses.

Example: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]] → [0,2,1,3]

```
def find_order(num_courses, prerequisites):
    graph = [[] for _ in range(num_courses)]
    in_degree = [0] * num_courses

    for course, prereq in prerequisites:
        graph[prereq].append(course)
        in_degree[course] += 1

    queue = deque([i for i in range(num_courses) if in_degree[i] == 0])
    result = []

    while queue:
        node = queue.popleft()
        result.append(node)
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    return result if len(result) == num_courses else []
```

Union Find Pattern

Union Find (Disjoint Set Union)

📖 Concept

Union Find (Disjoint Set Union) is a data structure that tracks elements partitioned into disjoint sets. It supports efficient union and find operations.

Key Idea: Use path compression and union by rank to achieve near-constant time operations. Perfect for connectivity problems, cycle detection, and grouping elements.

🎯 Techniques to Master

- **Path Compression:** Flatten tree during find operation for efficiency
- **Union by Rank:** Always attach smaller tree to larger tree
- **Find Operation:** Find root of element's set with path compression
- **Union Operation:** Merge two sets by connecting their roots
- **Connected Check:** Check if two elements are in same set
- **Count Components:** Track number of disjoint sets

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	$O(\alpha(n))$	α is inverse Ackermann function, effectively $O(1)$ for practical purposes
Space Complexity	$O(n)$	Space for parent and rank arrays

📚 Boilerplate Code

Union Find with Path Compression and Union by Rank:

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.components = n

    def find(self, x):
        """
        Find root with path compression
        """
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        """
        Union by rank
        """
        root_x = self.find(x)
        root_y = self.find(y)

        if root_x == root_y:
            return False  # Already connected

        # Union by rank
        if self.rank[root_x] < self.rank[root_y]:
            self.parent[root_x] = root_y
        elif self.rank[root_x] > self.rank[root_y]:
            self.parent[root_y] = root_x
        else:
            self.parent[root_y] = root_x
            self.rank[root_x] += 1

        self.components -= 1
        return True

    def connected(self, x, y):
        """
        Check if two elements are connected
        """
        return self.find(x) == self.find(y)

# TC: O(α(n)) per operation
# SC: O(n)
```

📝 Demonstrative Problems

Problem 1: Number of Connected Components

Description: Find number of connected components in an undirected graph.

Example: n = 5, edges = [[0,1],[1,2],[3,4]] → 2 components

```
def count_components(n, edges):
    uf = UnionFind(n)

    for u, v in edges:
        uf.union(u, v)

    return uf.components
```

Problem 2: Redundant Connection

Description: Find redundant edge that forms a cycle in a tree.

Example: edges = [[1,2],[1,3],[2,3]] → [2,3]

```
def find_redundant_connection(edges):
    n = len(edges)
    uf = UnionFind(n + 1)

    for u, v in edges:
        if not uf.union(u, v):
            return [u, v]  # Cycle detected

    return []
```

Problem 3: Number of Islands II

Description: Count islands after each land addition operation.

Example: m=3, n=3, positions=[[0,0],[0,1],[1,2],[2,1]] → [1,1,2,3]

```
def num_islands_2(m, n, positions):
    uf = UnionFind(m * n)
    result = []
    grid = [[0] * n for _ in range(m)]
    directions = [(0,1), (1,0), (0,-1), (-1,0)]

    for r, c in positions:
        if grid[r][c] == 1:
            result.append(uf.components)
            continue

        grid[r][c] = 1
        uf.components += 1
        idx = r * n + c

        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 ≤ nr < m and 0 ≤ nc < n and grid[nr][nc] == 1:
                neighbor_idx = nr * n + nc
                uf.union(idx, neighbor_idx)

        result.append(uf.components)

    return result
```

Two Heaps Pattern

Two Heaps

📖 Concept

The Two Heaps pattern uses a min-heap and max-heap to maintain elements in two halves, allowing efficient access to median, finding smallest/largest elements, or maintaining balanced partitions.

Key Idea: Use min-heap for larger half and max-heap for smaller half. Keep heaps balanced (size difference ≤ 1) for efficient median finding.

🎯 Techniques to Master

- **Min-Heap for Larger Half:** Store larger elements in min-heap
- **Max-Heap for Smaller Half:** Store smaller elements in max-heap (use negative values)
- **Balance Maintenance:** Keep heap sizes balanced (difference ≤ 1)
- **Median Finding:** Top of heaps give median when balanced
- **Re-balancing:** Move elements between heaps to maintain balance
- **Python heapq:** Use heapq (min-heap) with negatives for max-heap

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(log n)	Insert and rebalance operations take O(log n)
Space Complexity	O(n)	Store all elements in two heaps

💻 Boilerplate Code

Find Median from Data Stream:

```
import heapq

class MedianFinder:
    def __init__(self):
        # Max heap (smaller half) - use negative values
        self.small = []
        # Min heap (larger half)
        self.large = []

    def add_num(self, num):
        # Add to appropriate heap
        if not self.small or num ≤ -self.small[0]:
            heapq.heappush(self.small, -num)
        else:
            heapq.heappush(self.large, num)

        # Balance heaps
        if len(self.small) > len(self.large) + 1:
            val = -heapq.heappop(self.small)
            heapq.heappush(self.large, val)
        elif len(self.large) > len(self.small) + 1:
            val = heapq.heappop(self.large)
            heapq.heappush(self.small, -val)

    def find_median(self):
        if len(self.small) > len(self.large):
            return -self.small[0]
        elif len(self.large) > len(self.small):
            return self.large[0]
        else:
            return (-self.small[0] + self.large[0]) / 2.0

# TC: O(log n) per operation
# SC: O(n)
```

📝 Demonstrative Problems

Problem 1: Find Median from Data Stream

Description: Design a data structure that supports adding numbers and finding the median.

Example: addNum(1), addNum(2), findMedian() → 1.5, addNum(3), findMedian() → 2

```
class MedianFinder:
    def __init__(self):
        self.small = []  # Max heap (smaller half)
        self.large = []  # Min heap (larger half)

    def add_num(self, num):
        if not self.small or num ≤ -self.small[0]:
            heapq.heappush(self.small, -num)
        else:
            heapq.heappush(self.large, num)

        # Balance
        if len(self.small) > len(self.large) + 1:
            heapq.heappush(self.large, -heapq.heappop(self.small))
        elif len(self.large) > len(self.small) + 1:
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def find_median(self):
        if len(self.small) == len(self.large):
            return (-self.small[0] + self.large[0]) / 2.0
        return -self.small[0] if len(self.small) > len(self.large) else self.large[0]
```

Problem 2: Sliding Window Median

Description: Find median for each window of size k in an array.

Example: nums = [1,3,-1,-3,5,3,6,7], k = 3 → [1.0,-1.0,-1.0,3.0,5.0,6.0]

```
def median_sliding_window(nums, k):
    def get_median():
        if k % 2 == 1:
            return float(small[0][0] if len(small) > len(large) else -large[0][0])
        return (small[0][0] - large[0][0]) / 2.0

    small, large = [], []
    result = []

    # Initialize first window
    for i in range(k):
        heapq.heappush(small, (-nums[i], i))

    for i in range(k // 2):
        val, idx = heapq.heappop(small)
        heapq.heappush(large, (-val, idx))

    result.append(get_median())

    # Slide window
    for i in range(k, len(nums)):
        # Remove out-of-window elements
        while small and small[0][1] ≤ i - k:
            heapq.heappop(small)
        while large and large[0][1] ≤ i - k:
            heapq.heappop(large)

        # Add new element and balance
        if nums[i] ≤ -small[0][0] if small else True:
            heapq.heappush(small, (-nums[i], i))
        else:
            heapq.heappush(large, (nums[i], i))

        # Rebalance
        while len(small) > len(large) + 1:
            val, idx = heapq.heappop(small)
            heapq.heappush(large, (-val, idx))
        while len(large) > len(small) + 1:
            val, idx = heapq.heappop(large)
            heapq.heappush(small, (-val, idx))

        result.append(get_median())

    return result
```

Top K Elements Pattern

Top K Elements

📖 Concept

The Top K Elements pattern finds the K largest, smallest, or most frequent elements efficiently using heaps or quickselect.

Key Idea: Use min-heap of size K for largest elements, or max-heap for smallest elements. Alternatively, use quickselect for O(n) average case.

🎯 Techniques to Master

- **Min-Heap for K Largest:** Keep heap size K, pop smallest when full
- **Max-Heap for K Smallest:** Use negative values with min-heap
- **Frequency Counting:** Use Counter to track element frequencies
- **Quickselect:** Use partition algorithm for O(n) average case
- **Heap Size Management:** Maintain exactly K elements in heap
- **Custom Comparators:** Sort by frequency, value, or custom criteria

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(n log k)	Heap approach: n elements, heap size k
Space Complexity	O(k)	Heap stores at most k elements

💻 Boilerplate Code

Top K Largest Elements (Min-Heap):

```
import heapq

def top_k_largest(nums, k):
    """
    Find k largest elements using min-heap
    """
    heap = []

    for num in nums:
        heapq.heappush(heap, num)
        if len(heap) > k:
            heapq.heappop(heap)  # Remove smallest

    return heap

# TC: O(n log k)
# SC: O(k)
```

Top K Frequent Elements:

```
from collections import Counter

def top_k_frequent(nums, k):
    """
    Find k most frequent elements
    """
    count = Counter(nums)

    # Min-heap of size k
    heap = []
    for num, freq in count.items():
        heapq.heappush(heap, (freq, num))
        if len(heap) > k:
            heapq.heappop(heap)

    return [num for freq, num in heap]

# TC: O(n log k)
# SC: O(n)
```

📝 Demonstrative Problems

Problem 1: Kth Largest Element

Description: Find the kth largest element in an array.

Example: nums = [3,2,1,5,6,4], k = 2 → 5

```
def find_kth_largest(nums, k):
    heap = []

    for num in nums:
        heapq.heappush(heap, num)
        if len(heap) > k:
            heapq.heappop(heap)

    return heap[0]
```

Problem 2: Top K Frequent Elements

Description: Return k most frequent elements.

Example: nums = [1,1,1,2,2,3], k = 2 → [1,2]

```
def top_k_frequent(nums, k):
    count = Counter(nums)
    heap = []

    for num, freq in count.items():
        heapq.heappush(heap, (freq, num))
        if len(heap) > k:
            heapq.heappop(heap)

    return [num for freq, num in heap]
```

Problem 3: Top K Frequent Words

Description: Return k most frequent words, sorted by frequency then lexicographically.

Example: words = ["i","love","leetcode","i","love","coding"], k = 2 → ["i","love"]

```
def top_k_frequent_words(words, k):
    count = Counter(words)

    # Custom comparator: (-freq, word) for min-heap
    heap = []
    for word, freq in count.items():
        heapq.heappush(heap, (-freq, word))
        if len(heap) > k:
            heapq.heappop(heap)

    # Sort by frequency (desc) then word (asc)
    result = sorted(heap, key=lambda x: (x[0], x[1]))
    return [word for freq, word in result]
```

K-way Merge Pattern

K-way Merge

📖 Concept

K-way Merge combines K sorted arrays/lists into one sorted array efficiently using a min-heap to track the smallest element from each array.

Key Idea: Use a min-heap to always get the smallest element across all K arrays. Pop smallest, add next element from same array, repeat until all arrays are exhausted.

🎯 Techniques to Master

- **Min-Heap Approach:** Store (value, array_index, element_index) tuples
- **Pointer Tracking:** Track current position in each array
- **Heap Initialization:** Add first element from each array
- **Merge Process:** Pop smallest, add next from same array
- **Array Exhaustion:** Handle when an array is fully processed
- **Divide and Conquer:** Merge pairs recursively for large K

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(n log k)	n = total elements, k = number of arrays. Each element processed once, heap operations O(log k)
Space Complexity	O(k)	Heap stores at most k elements (one from each array)

📚 Boilerplate Code

Merge K Sorted Lists:

```
import heapq

def merge_k_lists(lists):
    """
    Merge k sorted linked lists
    """
    heap = []

    # Initialize heap with first node from each list
    for i, lst in enumerate(lists):
        if lst:
            heapq.heappush(heap, (lst.val, i, lst))

    dummy = ListNode(0)
    current = dummy

    while heap:
        val, list_idx, node = heapq.heappop(heap)
        current.next = node
        current = current.next

        # Add next node from same list
        if node.next:
            heapq.heappush(heap, (node.next.val, list_idx, node.next))

    return dummy.next

# TC: O(n log k)
# SC: O(k)
```

Merge K Sorted Arrays:

```
def merge_k_arrays(arrays):
    """
    Merge k sorted arrays
    """
    heap = []
    result = []

    # Initialize: (value, array_index, element_index)
    for i, arr in enumerate(arrays):
        if arr:
            heapq.heappush(heap, (arr[0], i, 0))

    while heap:
        val, arr_idx, elem_idx = heapq.heappop(heap)
        result.append(val)

        # Add next element from same array
        if elem_idx + 1 < len(arrays[arr_idx]):
            next_val = arrays[arr_idx][elem_idx + 1]
            heapq.heappush(heap, (next_val, arr_idx, elem_idx + 1))

    return result

# TC: O(n log k)
# SC: O(k)
```

📝 Demonstrative Problems

Problem 1: Merge K Sorted Lists

Description: Merge k sorted linked lists into one sorted list.

Example: lists = [[1,4,5],[1,3,4],[2,6]] → [1,1,2,3,4,4,5,6]

```
def merge_k_lists(lists):
    heap = []

    for i, lst in enumerate(lists):
        if lst:
            heapq.heappush(heap, (lst.val, i, lst))

    dummy = ListNode(0)
    current = dummy

    while heap:
        val, idx, node = heapq.heappop(heap)
        current.next = node
        current = current.next

        if node.next:
            heapq.heappush(heap, (node.next.val, idx, node.next))

    return dummy.next
```

Problem 2: Kth Smallest in Sorted Matrix

Description: Find kth smallest element in a sorted matrix.

Example: matrix = [[1,5,9],[10,11,13],[12,13,15]], k = 8 → 13

```
def kth_smallest(matrix, k):
    heap = []
    n = len(matrix)

    # Add first element of each row
    for i in range(n):
        heapq.heappush(heap, (matrix[i][0], i, 0))

    for _ in range(k - 1):
        val, row, col = heapq.heappop(heap)
        if col + 1 < n:
            heapq.heappush(heap, (matrix[row][col + 1], row, col + 1))

    return heap[0][0]
```

Trie Pattern

Trie (Prefix Tree)

📖 Concept

A Trie (prefix tree) is a tree-like data structure for storing strings. Each node represents a character, and paths from root to nodes represent words.

Key Idea: Share common prefixes among words. Efficient for prefix matching, autocomplete, word search, and dictionary operations.

🎯 Techniques to Master

- **Node Structure:** Each node has children (dict) and is_end flag
- **Insert Operation:** Traverse/create path character by character
- **Search Operation:** Check if word exists in trie
- **Prefix Search:** Check if prefix exists
- **DFS Traversal:** Find all words with given prefix
- **Space Optimization:** Use array of size 26 for lowercase letters

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(m)	m = length of word. Each operation traverses word length
Space Complexity	O(ALPHABET_SIZE * N * M)	N = number of words, M = average length. Can be optimized

📚 Boilerplate Code

Basic Trie Implementation:

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        """
        Insert word into trie
        """
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end = True

    def search(self, word):
        """
        Check if word exists in trie
        """
        node = self.root
        for char in word:
            if char not in node.children:
                return False
            node = node.children[char]
        return node.is_end

    def starts_with(self, prefix):
        """
        Check if prefix exists
        """
        node = self.root
        for char in prefix:
            if char not in node.children:
                return False
            node = node.children[char]
        return True

# TC: O(m) per operation
# SC: O(ALPHABET_SIZE * N * M)
```

📝 Demonstrative Problems

Problem 1: Implement Trie

Description: Implement a trie with insert, search, and startsWith methods.

Example: insert("apple"), search("apple") → True, startsWith("app") → True

```
class Trie:
    def __init__(self):
        self.root = {}

    def insert(self, word):
        node = self.root
        for char in word:
            if char not in node:
                node[char] = {}
            node = node[char]
        node['#'] = True  # Mark end of word

    def search(self, word):
        node = self.root
        for char in word:
            if char not in node:
                return False
            node = node[char]
        return '#' in node

    def starts_with(self, prefix):
        node = self.root
        for char in prefix:
            if char not in node:
                return False
            node = node[char]
        return True
```

Problem 2: Word Search II

Description: Find all words from dictionary that exist in the board.

Example: board = [["o","a","a","n"],["e","t","a","e"],["i","h","k","r"],["i","f","l","v"]], words = ["oath","pea","eat","rain"] → ["eat","oath"]

```
def find_words(board, words):
    trie = {}
    for word in words:
        node = trie
        for char in word:
            node = node.setdefault(char, {})
        node['#'] = word

    result = []

    def dfs(r, c, node):
        char = board[r][c]
        curr_node = node[char]

        word_match = curr_node.pop('#', False)
        if word_match:
            result.append(word_match)

        board[r][c] = '#'
        for dr, dc in [(0,1), (1,0), (0,-1), (-1,0)]:
            nr, nc = r + dr, c + dc
            if 0 ≤ nr < len(board) and 0 ≤ nc < len(board[0]) and board[nr][nc] in curr_node:
                dfs(nr, nc, curr_node)
        board[r][c] = char

        if not curr_node:
            node.pop(char)

    for i in range(len(board)):
        for j in range(len(board[0])):
            if board[i][j] in trie:
                dfs(i, j, trie)

    return result
```

Modified Binary Search Pattern

Modified Binary Search

📖 Concept

Modified Binary Search adapts the classic binary search for problems like finding boundaries, searching in rotated arrays, or finding optimal values.

Key Idea: Instead of searching for exact match, search for first/last occurrence, boundary conditions, or optimal value using binary search on answer space.

🎯 Techniques to Master

- **Search Space Reduction:** Narrow down search space based on conditions
- **Boundary Finding:** Find first/last position where condition is true
- **Rotated Array Search:** Determine which half is sorted
- **Binary Search on Answer:** Search for optimal value in answer space
- **Condition Functions:** Define clear true/false conditions
- **Left/Right Boundaries:** Use left <= right vs left < right appropriately

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(log n)	Each iteration eliminates half of search space
Space Complexity	O(1)	Only using constant extra space

📚 Boilerplate Code

Find First True (Lower Bound):

```
def find_first_true(arr, condition):
    """
    Find first index where condition is true
    """
    left, right = 0, len(arr)

    while left < right:
        mid = (left + right) // 2
        if condition(arr[mid]):
            right = mid
        else:
            left = mid + 1

    return left

# TC: O(log n)
# SC: O(1)
```

Search in Rotated Sorted Array:

```
def search_rotated(nums, target):
    """
    Search in rotated sorted array
    """
    left, right = 0, len(nums) - 1

    while left ≤ right:
        mid = (left + right) // 2

        if nums[mid] == target:
            return mid

        # Left half is sorted
        if nums[left] ≤ nums[mid]:
            if nums[left] ≤ target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        # Right half is sorted
        else:
            if nums[mid] < target ≤ nums[right]:
                left = mid + 1
            else:
                right = mid - 1

    return -1

# TC: O(log n)
# SC: O(1)
```

Binary Search on Answer:

```
def binary_search_answer(min_val, max_val, condition):
    """
    Binary search on answer space
    """
    left, right = min_val, max_val
    result = -1

    while left ≤ right:
        mid = (left + right) // 2
        if condition(mid):
            result = mid
            right = mid - 1  # or left = mid + 1 depending on problem
        else:
            left = mid + 1  # or right = mid - 1

    return result

# TC: O(log(max_val - min_val))
# SC: O(1)
```

📝 Demonstrative Problems

Problem 1: Search in Rotated Sorted Array

Description: Search target in rotated sorted array with no duplicates.

Example: nums = [4,5,6,7,0,1,2], target = 0 → 4

```
def search(nums, target):
    left, right = 0, len(nums) - 1

    while left ≤ right:
        mid = (left + right) // 2

        if nums[mid] == target:
            return mid

        if nums[left] ≤ nums[mid]:
            if nums[left] ≤ target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:
            if nums[mid] < target ≤ nums[right]:
                left = mid + 1
            else:
                right = mid - 1

    return -1
```

Problem 2: Find Peak Element

Description: Find peak element (greater than neighbors) in array.

Example: nums = [1,2,3,1] → 2 (index of 3)

```
def find_peak_element(nums):
    left, right = 0, len(nums) - 1

    while left < right:
        mid = (left + right) // 2
        if nums[mid] > nums[mid + 1]:
            right = mid
        else:
            left = mid + 1

    return left
```

Problem 3: Search a 2D Matrix

Description: Search target in 2D matrix where each row and column is sorted.

Example: matrix = [[1,4,7,11],[2,5,8,12],[3,6,9,16],[10,13,14,17]], target = 5 → True

```
def search_matrix(matrix, target):
    if not matrix:
        return False

    row, col = 0, len(matrix[0]) - 1

    while row < len(matrix) and col ≥ 0:
        if matrix[row][col] == target:
            return True
        elif matrix[row][col] > target:
            col -= 1
        else:
            row += 1

    return False
```

Subsets Pattern

Subsets (Backtracking)

📖 Concept

The Subsets pattern generates all possible subsets/combinations/permutations using backtracking. Each element can be included or excluded.

Key Idea: Use recursion to explore all possibilities. At each step, make a choice (include/exclude), recurse, then backtrack to try other choices.

🎯 Techniques to Master

- **Backtracking Template:** Choose, explore, unchoose pattern
- **Include/Exclude:** Two choices at each step
- **Duplicate Handling:** Sort and skip duplicates
- **Path Tracking:** Maintain current path/combination
- **Base Case:** When to add current path to result
- **Pruning:** Skip invalid paths early

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(2^n)	For subsets: 2^n possible subsets (each element included/excluded)
Space Complexity	O(n)	Recursion stack depth and path storage

📚 Boilerplate Code

Generate All Subsets:

```
def subsets(nums):
    """
    Generate all subsets
    """
    result = []

    def backtrack(start, path):
        # Add current path to result
        result.append(path[:])

        # Try each remaining element
        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()  # Backtrack

    backtrack(0, [])
    return result

# TC: O(2^n)
# SC: O(n)
```

Subsets with Duplicates:

```
def subsets_with_dup(nums):
    """
    Generate subsets, skip duplicates
    """
    nums.sort()
    result = []

    def backtrack(start, path):
        result.append(path[:])

        for i in range(start, len(nums)):
            # Skip duplicates
            if i > start and nums[i] == nums[i - 1]:
                continue

            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()

    backtrack(0, [])
    return result

# TC: O(2^n)
# SC: O(n)
```

Combinations:

```
def combine(n, k):
    """
    Generate all combinations of k numbers from [1, n]
    """
    result = []

    def backtrack(start, path):
        if len(path) == k:
            result.append(path[:])
            return

        for i in range(start, n + 1):
            path.append(i)
            backtrack(i + 1, path)
            path.pop()

    backtrack(1, [])
    return result

# TC: O(C(n,k))
# SC: O(k)
```

📝 Demonstrative Problems

Problem 1: Subsets

Description: Return all possible subsets of an array.

Example: nums = [1,2,3] → [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

```
def subsets(nums):
    result = []

    def backtrack(start, path):
        result.append(path[:])
        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()

    backtrack(0, [])
    return result
```

Problem 2: Subsets II

Description: Generate subsets with duplicates, no duplicate subsets.

Example: nums = [1,2,2] → [[],[1],[1,2],[1,2,2],[2],[2,2]]

```
def subsets_with_dup(nums):
    nums.sort()
    result = []

    def backtrack(start, path):
        result.append(path[:])
        for i in range(start, len(nums)):
            if i > start and nums[i] == nums[i - 1]:
                continue
            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()

    backtrack(0, [])
    return result
```

Problem 3: Combination Sum

Description: Find all unique combinations that sum to target (can reuse elements).

Example: candidates = [2,3,6,7], target = 7 → [[2,2,3],[7]]

```
def combination_sum(candidates, target):
    result = []

    def backtrack(start, path, remaining):
        if remaining == 0:
            result.append(path[:])
            return
        if remaining < 0:
            return

        for i in range(start, len(candidates)):
            path.append(candidates[i])
            backtrack(i, path, remaining - candidates[i])
            path.pop()

    backtrack(0, [], target)
    return result
```

Bitwise XOR Pattern

Bitwise XOR

📖 Concept

Bitwise XOR (^) has unique properties that make it useful for solving problems involving finding unique elements, swapping without temp, or detecting differences.

Key Properties: a ^ a = 0, a ^ 0 = a, a ^ b = b ^ a, (a ^ b) ^ c = a ^ (b ^ c). XOR of all elements cancels pairs.

🎯 Techniques to Master

- **Find Single Number:** XOR all numbers, pairs cancel out
- **Swap Without Temp:** a ^= b; b ^= a; a ^= b
- **Find Missing Number:** XOR all numbers with range
- **Find Two Unique Numbers:** XOR all, find rightmost set bit, partition
- **Complement Property:** Use XOR with mask to flip bits
- **Parity Checking:** XOR to check if count is odd/even

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(n)	Single pass through array
Space Complexity	O(1)	Only using constant extra space

💻 Boilerplate Code

Find Single Number:

```
def single_number(nums):
    """
    Find single number in array where all others appear twice
    """
    result = 0
    for num in nums:
        result ^= num
    return result

# TC: O(n)
# SC: O(1)
```

Find Two Single Numbers:

```
def single_numbers(nums):
    """
    Find two numbers that appear once, others appear twice
    """
    # XOR all numbers
    xor_all = 0
    for num in nums:
        xor_all ^= num

    # Find rightmost set bit
    rightmost_bit = xor_all & -xor_all

    # Partition into two groups
    num1 = num2 = 0
    for num in nums:
        if num & rightmost_bit:
            num1 ^= num
        else:
            num2 ^= num

    return [num1, num2]

# TC: O(n)
# SC: O(1)
```

Swap Without Temporary Variable:

```
def swap(a, b):
    """
    Swap two numbers using XOR
    """
    a ^= b
    b ^= a
    a ^= b
    return a, b

# TC: O(1)
# SC: O(1)
```

📝 Demonstrative Problems

Problem 1: Single Number

Description: Find the single number that appears once, others appear twice.

Example: nums = [2,2,1] → 1

```
def single_number(nums):
    result = 0
    for num in nums:
        result ^= num
    return result
```

Problem 2: Single Number III

Description: Find two numbers that appear once, others appear twice.

Example: nums = [1,2,1,3,2,5] → [3,5]

```
def single_number_3(nums):
    xor_all = 0
    for num in nums:
        xor_all ^= num

    # Get rightmost set bit
    diff = xor_all & -xor_all

    num1 = num2 = 0
    for num in nums:
        if num & diff:
            num1 ^= num
        else:
            num2 ^= num

    return [num1, num2]
```

Problem 3: Missing Number

Description: Find missing number in array containing n distinct numbers from [0, n].

Example: nums = [3,0,1] → 2

```
def missing_number(nums):
    result = len(nums)
    for i, num in enumerate(nums):
        result ^= i ^ num
    return result
```

Dynamic Programming Pattern

Dynamic Programming

📖 Concept

Dynamic Programming solves complex problems by breaking them into simpler subproblems and storing results to avoid recomputation. It optimizes recursive solutions with overlapping subproblems.

Key Idea: Identify optimal substructure (optimal solution contains optimal solutions to subproblems) and overlapping subproblems. Use memoization (top-down) or tabulation (bottom-up).

🎯 Techniques to Master

- **Memoization (Top-Down):** Cache results of recursive calls
- **Tabulation (Bottom-Up):** Build solution iteratively from base cases
- **1D DP:** dp[i] represents state at position i
- **2D DP:** dp[i][j] represents state with two dimensions
- **State Transition:** Define how to compute current state from previous states
- **Space Optimization:** Reduce space by reusing arrays
- **Pattern Recognition:** Identify DP patterns (knapsack, LIS, edit distance, etc.)

⏱️ Time & Space Complexity

Complexity Type	Value	Explanation
Time Complexity	O(n) to O(n^2)	Depends on problem. Typically O(states × transitions)
Space Complexity	O(n) to O(n^2)	Space for DP table. Can often be optimized to O(1) or O(n)

💻 Boilerplate Code

1D DP (Fibonacci):

```
def fibonacci(n):
    """
    Bottom-up DP approach
    """
    if n ≤ 1:
        return n

    dp = [0] * (n + 1)
    dp[1] = 1

    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp[n]

# Space optimized
def fibonacci_optimized(n):
    if n ≤ 1:
        return n

    prev2, prev1 = 0, 1
    for i in range(2, n + 1):
        curr = prev1 + prev2
        prev2, prev1 = prev1, curr

    return prev1

# TC: O(n)
# SC: O(1) optimized
```

2D DP (Unique Paths):

```
def unique_paths(m, n):
    """
    Number of unique paths from top-left to bottom-right
    """
    dp = [[1] * n for _ in range(m)]

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1]

    return dp[m - 1][n - 1]

# Space optimized
def unique_paths_optimized(m, n):
    prev = [1] * n

    for i in range(1, m):
        curr = [1] * n
        for j in range(1, n):
            curr[j] = prev[j] + curr[j - 1]
        prev = curr

    return prev[n - 1]

# TC: O(m * n)
# SC: O(n) optimized
```

Memoization (Top-Down):

```
def coin_change_memo(coins, amount):
    """
    Minimum coins to make amount (memoization)
    """
    memo = {}

    def dp(remaining):
        if remaining == 0:
            return 0
        if remaining < 0:
            return float('inf')
        if remaining in memo:
            return memo[remaining]

        min_coins = float('inf')
        for coin in coins:
            result = dp(remaining - coin)
            if result ≠ float('inf'):
                min_coins = min(min_coins, 1 + result)

        memo[remaining] = min_coins
        return min_coins

    result = dp(amount)
    return result if result ≠ float('inf') else -1

# TC: O(amount * len(coins))
# SC: O(amount)
```

📝 Demonstrative Problems

Problem 1: Climbing Stairs

Description: Find number of ways to climb n stairs (1 or 2 steps at a time).

Example: n = 3 → 3 ways (1+1+1, 1+2, 2+1)

```
def climb_stairs(n):
    if n ≤ 2:
        return n

    prev2, prev1 = 1, 2
    for i in range(3, n + 1):
        curr = prev1 + prev2
        prev2, prev1 = prev1, curr

    return prev1
```

Problem 2: Coin Change

Description: Find minimum coins to make amount.

Example: coins = [1,2,5], amount = 11 → 3 (5+5+1)

```
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for i in range(1, amount + 1):
        for coin in coins:
            if i ≥ coin:
                dp[i] = min(dp[i], dp[i - coin] + 1)

    return dp[amount] if dp[amount] ≠ float('inf') else -1
```

Problem 3: Longest Increasing Subsequence

Description: Find length of longest strictly increasing subsequence.

Example: nums = [10,9,2,5,3,7,101,18] → 4 ([2,3,7,101])

```
def length_of_lis(nums):
    dp = [1] * len(nums)

    for i in range(1, len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)

    return max(dp)
```

BST & Spatial Trees (KD-Tree) Pattern

Binary Search Tree (BST)

📖 Concept

A Binary Search Tree is a binary tree where for every node: **left subtree < root < right subtree**. In-order traversal yields sorted order. Used for fast search, insert, delete (average O(log n)) and range/successor queries.

Key Idea: Use the ordering property to discard one subtree per step. Validate by passing (min, max) bounds down. Kth smallest = in-order with a counter.

🎯 Techniques to Master

- **Search:** Compare with root, recurse left or right
- **Insert:** Find leaf position by comparing, then attach
- **Delete:** 0 children → remove; 1 child → bypass; 2 children → replace with inorder successor/predecessor
- **Validate BST:** Pass (min, max) bounds; left must be in (min, root.val), right in (root.val, max)
- **Kth smallest:** In-order traversal with counter; return when count == k
- **BST from sorted array:** Mid as root, recurse left/right on halves
- **Successor / Predecessor:** Use parent pointers or in-order stack

⏱️ Time & Space Complexity (BST)

Operation	Time (avg)	Time (worst)	Space
Search / Insert / Delete	O(log n)	O(n)	O(h)
In-order (sorted)	O(n)	O(n)	O(h)
Validate / Kth smallest	O(n)	O(n)	O(h)
h = height; balanced BST gives h = O(log n)			

🧱 BST Boilerplate

Search & Validate BST:

```
def search_bst(root, target):
    if not root or root.val == target:
        return root
    if target < root.val:
        return search_bst(root.left, target)
    return search_bst(root.right, target)

def is_valid_bst(root, lo=float('-inf'), hi=float('inf')):
    if not root:
        return True
    if not (lo < root.val < hi):
        return False
    return is_valid_bst(root.left, lo, root.val) and is_valid_bst(root.right, root.val, hi)

# TC: Search O(h), Validate O(n). SC: O(h)
```

Kth Smallest (in-order):

```
def kth_smallest(root, k):
    def inorder(node):
        nonlocal k, ans
        if not node or ans is not None:
            return
        inorder(node.left)
        k -= 1
        if k == 0:
            ans = node.val
            return
        inorder(node.right)
    ans = None
    inorder(root)
    return ans

# TC: O(k) to O(n). SC: O(h)
```

BST from Sorted Array:

```
def sorted_array_to_bst(nums):
    def build(l, r):
        if l > r:
            return None
        mid = (l + r) // 2
        node = TreeNode(nums[mid])
        node.left = build(l, mid - 1)
        node.right = build(mid + 1, r)
        return node
    return build(0, len(nums) - 1)

# TC: O(n). SC: O(log n) recursion
```

📝 BST Problems

Problem 1: Validate BST

Description: Check if binary tree is a valid BST (strict min/max per node).

```
def isValidBST(root):
    def valid(node, lo, hi):
        if not node: return True
        if not (lo < node.val < hi): return False
        return valid(node.left, lo, node.val) and valid(node.right, node.val, hi)
    return valid(root, float('-inf'), float('inf'))
```

Problem 2: Kth Smallest in BST

Description: Return the kth smallest value (1-indexed) using in-order.

```
def kthSmallest(root, k):
    stack = []
    while root or stack:
        while root:
            stack.append(root)
            root = root.left
        root = stack.pop()
        k -= 1
        if k == 0:
            return root.val
        root = root.right
```

KD-Tree (K-Dimensional Tree)

📖 Concept

A KD-tree is a space-partitioning tree for organizing points in **k-dimensional** space. Each level splits along one dimension (cycling through dimensions). Used for **nearest neighbor search**, range queries, and spatial indexing.

Key Idea: Build by repeatedly splitting on median along alternating axes. Search: traverse like BST (compare on current axis), then possibly check the other subtree if the splitting hyperplane is within the search radius.

🎯 Techniques to Master

- **Build:** Choose dimension (e.g. depth % k), find median, split into left/right; recurse
- **Nearest Neighbor:** Go toward query point first; then backtrack to the "other side" if the hyperplane is closer than best so far
- **Range Query:** Check if node's point in range; recurse left/right if the splitting plane intersects the range
- **Dimension cycling:** Level 0 → axis 0, level 1 → axis 1, … level k → axis 0 again

⏱️ Time & Space (KD-Tree)

Operation	Time (avg)	Time (worst)	Space
Build (n points)	O(n log n)	$O(n^2)$	O(n)
Nearest neighbor	O(log n)	O(n)	O(log n)
Range query	$O(\sqrt{n} + k)$	O(n)	O(log n)
k = number of points in range; worst case in high dimensions			

🟫 KD-Tree Boilerplate (2D)

```
class KDNode:
    def __init__(self, point, left=None, right=None):
        self.point = point
        self.left = left
        self.right = right

def build_kdtree(points, depth=0):
    if not points:
        return None
    k = len(points[0])
    axis = depth % k
    points_sorted = sorted(points, key=lambda p: p[axis])
    mid = len(points_sorted) // 2
    return KDNode(
        points_sorted[mid],
        build_kdtree(points_sorted[:mid], depth + 1),
        build_kdtree(points_sorted[mid + 1:], depth + 1)
    )

def dist_sq(p, q):
    return sum((a - b) ** 2 for a, b in zip(p, q))

def nearest(node, query, depth=0, best=None):
    if not node:
        return best
    k = len(query)
    axis = depth % k
    d = dist_sq(node.point, query)
    if best is None or d < dist_sq(best, query):
        best = node.point
    if query[axis] < node.point[axis]:
        near, far = node.left, node.right
    else:
        near, far = node.right, node.left
    best = nearest(near, query, depth + 1, best)
    # Check other side if hyperplane is within current best distance
    if far and (query[axis] - node.point[axis]) ** 2 < dist_sq(best, query):
        best = nearest(far, query, depth + 1, best)
    return best

# Build: O(n log n). Nearest: O(log n) average. SC: O(n)
```

📝 KD-Tree Problem

Problem: K Nearest Neighbors (KNN) with KD-Tree

Description: Given points and a query, find K nearest points. Maintain a max-heap of size K (or collect and sort) while doing nearest-neighbor-style traversal.

```
import heapq
def k_nearest_kdtree(root, query, k):
    heap = []  # max-heap by negative distance
    def search(node, depth=0):
        if not node:
            return
        axis = depth % len(query)
        d = dist_sq(node.point, query)
        if len(heap) < k:
            heapq.heappush(heap, (-d, node.point))
        elif d < -heap[0][0]:
            heapq.heapreplace(heap, (-d, node.point))
        if query[axis] < node.point[axis]:
            search(node.left, depth + 1)
            if len(heap) < k or (node.point[axis] - query[axis])**2 ≤ -heap[0][0]:
                search(node.right, depth + 1)
        else:
            search(node.right, depth + 1)
            if len(heap) < k or (query[axis] - node.point[axis])**2 ≤ -heap[0][0]:
                search(node.left, depth + 1)
    search(root)
    return [p for _, p in heap]

# TC: O(log n + k) average. SC: O(k + log n)
```

Other Spatial / Partitioning Trees

📖 Overview

R-tree: B-tree-like structure for rectangles (bounding boxes). Good for geographic data, "find all objects in this region." Insert/search by merging/splitting bounding boxes.

BSP (Binary Space Partitioning): Recursively split space with hyperplanes. Used in graphics (visibility) and collision detection.

Quadtree / Octree: Split 2D/3D space into 4/8 children at each node. Good for grid-like data, collision, LOD. Simpler than KD-tree for axis-aligned subdivisions.

When to use: KD-tree → nearest neighbor in moderate dimensions; R-tree → range queries on rectangles; Quadtree → 2D grids and games.